

Pintos 2 Final Report

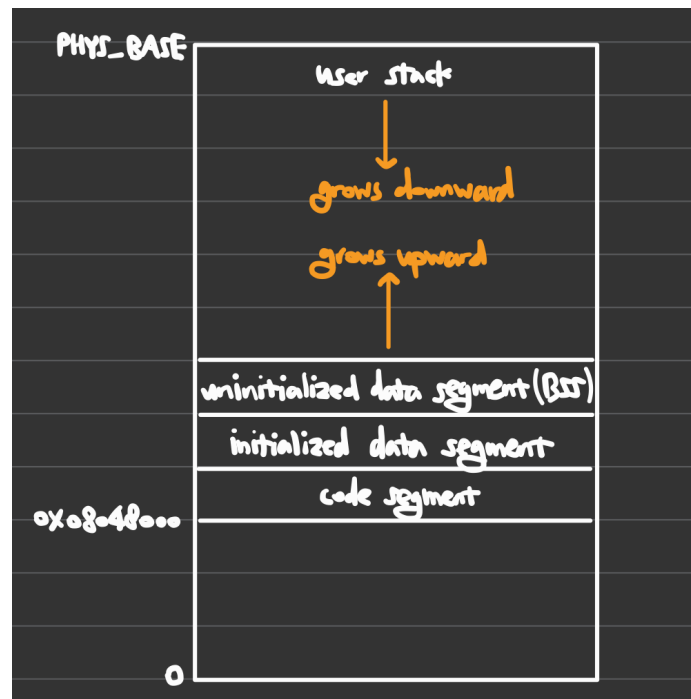
Team name	한라봉	
Team info.	20210807 이세광	20210750 이명률

0. Prior knowledge

1) Virtual memory layout

Pintos에서 virtual memory(4GB)는 user/kernel의 두 영역으로 나뉜다. User virtual memory는 virtual address 0에서부터 PHYS_BASE(3GB, 0xc0000000)까지의 범위이고, kernel virtual memory는 virtual address space의 나머지 부분인 PHYS_BASE부터 4GB를 점유한다. User virtual memory는 process별로 독립적이며, process가 전환될 때 page directory register가 바뀐다. Kernel virtual memory는 전역적이며 모든 process/thread가 공유한다. 또한 physical memory와 1:1로 매핑된다. 접근 권한의 경우, user program이 kernel 영역 (PHYS_BASE 이상)에 접근할 경우 page fault가 발생하며 process가 종료된다.

User stack은 PHYS_BASE 바로 아래에서 시작하여 낮은 주소(아래)로 자란다. 코드, 초기화된 데이터, 초기화되지 않은 데이터(BSS) segment는 낮은 주소에 배치되어 높은 주소(위)로 자란다.



1. Analysis of process execution procedure

1) threads/init.c

init.c 파일은 pintos kernel의 시작점이다. 컴퓨터가 부팅되고 pintos kernel이 memory에 로드되면 가장 먼저 main() 함수가 실행된다. main() 함수는 운영체제에 필요한 모든 핵심 요소들(memory 관리, thread, interrupt 등)을 순서대로 초기화하고, 마지막으로 사용자가 실행하라고 명령한 프로그램을 실행시키는 역할을 한다.

- main(): 시스템 부팅 및 초기화를 진행한다.
 - 초기화 단계: main 함수는 BSS segment 초기화, 메모리 관리자(palloc, malloc, paging) 초기화, thread 시스템(thread_init) 초기화, interrupt handler(intr_init) 및 각종 장치 드라이버 초기화를 수행한다.
 - #ifdef USERPROG: pintos project 2(userprog)부터 활성화되는 부분으로, user process를 kernel과 분리하기 위한 TSS와 GDT를 설정하고 system call과 예외 처리를 위한 syscall_init(), exception_init()을 호출한다.
 - Scheduler 시작: thread_start() 함수를 통해 thread scheduler가 동작을 시작하고 multitasking이 가능해진다.
 - run_actions(argv) 호출: 모든 시스템 초기화가 완료된 후, main 함수는 command 라인에서 받은 인자(argv)를 run_actions 함수에 넘겨주어 실제 작업을 수행하도록 한다. Process 실행은 이 함수로부터 시작된다.
- run_actions(): command line 작업을 실행한다. (pintos 명령어와 함께 주어진 action을 찾아 실행한다 – 예를 들어 pintos -- -q run alarm-multiple이라는 명령어가 있다면 run이 action에 해당한다)
- run_task(): 특정 작업(process) 실행을 요청한다. run action에 대한 실질적인 처리를 담당하며, 가장 핵심적인 부분은 process_wait(process_execute(task));이다. Pintos project 2(userprog)부터 이 코드가 실행된다.

2) userprog/process.c

- process_execute(): user program을 실행하기 위한 새로운 thread를 생성한다.
- start_process(): process_execute()에 의해 생성된 새로운 kernel thread는 이 함수에서 실행을 시작한다. 이 함수의 최종 목표는 kernel mode에서 user mode로 전환하여 user program을 실행시키는 것이다.
- load(): LF(Executable and Linkable Format) 형식의 실행 파일을 파싱하고, 파일의 내용을 virtual memory에 올리는 복잡한 작업을 수행한다.

3) How process execution works in the current pintos system?

① Kernel 부팅 및 실행 준비 (init.c)

- A. pintos kernel이 시작되면 threads/init.c의 main() 함수가 가장 먼저 실행된다.
- B. main() 함수는 메모리 관리 시스템(palloc_init, paging_init), thread 시스템(thread_init), interrupt handler(intr_init) 등 kernel의 핵심 기능들을 초기화한다.
- C. #ifdef USERPROG 블록 안에서 user process를 지원하기 위한 GDT(gdt_init), TSS(tss_init), system call handler(syscall_init) 등을 초기화한다.
- D. read_command_line()과 parse_options()를 통해 사용자가 pintos를 실행할 때 입력한 명령어를 분석한다.
- E. 모든 초기화가 끝나면, main() 함수는 run_actions()를 호출한다. run_actions()는 run이라는 명령어를 발견하고, 이때 매핑된 run_task() 함수를 호출한다.

② Process 생성 요청 (init.c → process.c)

- A. init.c의 run_task() 함수는 실행할 프로그램의 이름을 인자로 받는다.
- B. run_task()는 userprog/process.c에 정의된 process_execute() 함수를 호출하여 실질적인 process 생성을 요청한다. 밑의 코드 한 줄이 모든 process 실행의 시작점이다:

```
1. // in threads/init.c, run_task()  
2. process_wait (process_execute (task));
```

③ 새로운 thread 생성 및 프로그램 load 준비 (process.c)

- A. process_execute():
 - i. file_name의 복사본을 만들어 경쟁 상태를 방지한다.
 - ii. thread_create()를 호출하여 새로운 kernel thread를 생성한다. 이 thread는 user program을 실행할 주체가 된다. 이때 새로 생성된 thread가 실행을 시작할 함수로 start_process()를 지정하고, 인자로 프로그램 이름 복사본을 넘겨준다.
 - iii. process_execute()는 즉시 새로 생성된 thread의 ID(tid)를 반환한다.
- B. start_process():
 - i. 새로 생성된 kernel thread는 이 함수에서 실행을 시작한다.
 - ii. 먼저 load() 함수를 호출하여 disk에 있는 실행 파일을 메모리에 올린다.
 - iii. load() 함수가 성공적으로 완료되면, 프로그램의 시작 주소(eip)와 user stack pointer(esp) 값을 얻게 된다.

④ ELF 파일 load 및 virtual memory 설정 (process.c의 load() 함수)

- A. 각 process는 독립적인 virtual memory space를 가진다. pagedir_create()를 호출하여 이 process만을 위한 새로운 page directory를 생성하고 활성화한다.
- B. 실행 파일을 열고, 이것이 유효한 ELF 바이너리인지 header 정보를 읽어 검증한

다.

- C. ELF 파일의 프로그램 header를 읽어 memory에 적재해야 할 segment(code, data 등)를 확인한다.
- D. load_segment()를 통해 각 segment의 크기만큼 physical memory page(palloc_get_page)를 할당받는다. 이후 파일에서 내용을 읽어 채우고, 이 physical page를 process의 virtual memory space에 매핑(install_page)한다.
- E. setup_stack()을 통해 user stack으로 사용할 physical memory page 1개를 할당하고, virtual address space의 최상단에 매핑한다.

⑤ User mode로 전환 및 프로그램 실행 (process.c)

- A. load() 함수가 성공적으로 return되면 start_process() 함수는 프로그램의 시작 주소(eip)와 stack pointer(esp)를 intr_frame 구조체에 저장한다.
- B. start_process()의 마지막 부분에 있는 inline assembly code가 실행된다.

```
1. // in userprog/process.c, start_process()
2. asm volatile ("movl %0, %%esp; jmp intr_exit" : : "g" (&if_) : "memory");
```

- i. 위의 assembly code는 interrupt에서 복귀하는 과정을 흉내낸다. stack pointer가 intr_frame을 가리키게 한 후 intr_exit으로 점프하면, iret 명령이 실행되면서 CPU는 kernel mode에서 user mode로 전환된다.
 - ii. 이때 intr_frame에 저장해두었던 값들이 CPU register로 복원되면서, instruction pointer(EIP)는 프로그램의 시작 주소를, stack pointer(esp)는 user stack을 가리키게 된다.
- C. 위의 과정을 통해, CPU는 user program의 첫 번째 기계어 명령부터 실행을 시작한다.

⑥ Parent process의 대기 및 시스템 종료 (init.c)

- A. Child process가 위의 ③~⑤단계를 거쳐 실행되는 동안, parent process(최초의 main thread)는 ②단계에서 호출했던 process_wait() 함수 안에서 child process가 종료될 때까지 대기한다.
- B. User program 실행이 모두 끝나고 exit system call을 호출하면, child process는 process_exit()을 통해 모든 자원을 해제하고 종료된다. 이때 process_wait()에서 대기하던 parent가 깨어나고 run_task() 함수가 return된다.
- C. 모든 run action이 끝나면 main 함수는 shutdown()을 호출하여 pintos를 종료한다.

위의 과정을 요약하면 다음과 같이 작성해볼 수 있다:

init.c: main() → init.c: run_actions() → init.c: run_task()

→ process.c: process_execute() (새 thread 생성) → (new thread) → process.c:

start_process() → process.c: load() (ELF 파일 load 및 memory 매핑) → process.c:
start_process() (user mode로 전환) → (user program 실행) → (프로그램 종료) →
process.c: process_exit()

→ init.c: process_wait() return → init.c: shutdown()

2. Analysis of system call procedure

1) lib/user/syscall.c

이 파일은 user program의 영역에서 동작하는 코드이다. 즉 kernel code가 아니라, user program이 kernel의 기능을 사용하기 위해 호출하는 C 표준 라이브러리 함수들의 pintos 구현체이다. User program은 파일 열기, 화면에 글자 출력, process 종료 등 운영 체제의 보호된 기능이 필요할 때 직접 kernel code에 접근할 수 없으며, 대신 이 파일에 정의된 함수를 통해 kernel에게 해당 서비스를 요청하게 된다. 이 파일의 모든 system call 함수들(halt, exit, write 등)이 다음과 같이 정의되어 있고, 이 system call 함수들은 내부적으로 syscall0부터 syscall3까지의 macro 중 하나를 호출한다:

```
1. void
2. halt (void)
3. {
4.     syscall0 (SYS_HALT);
5.     NOT_REACHED ();
6. }
7.
8. void
9. exit (int status)
10. {
11.     syscall1 (SYS_EXIT, status);
12.     NOT_REACHED ();
13. }
14.
15. pid_t
16. exec (const char *file)
17. {
18.     return (pid_t) syscall1 (SYS_EXEC, file);
19. }
20.
21. int
22. wait (pid_t pid)
23. {
24.     return syscall1 (SYS_WAIT, pid);
25. }
26.
27. bool
28. create (const char *file, unsigned initial_size)
29. {
30.     return syscall2 (SYS_CREATE, file, initial_size);
31. }
32.
33. bool
34. remove (const char *file)
35. {
36.     return syscall1 (SYS_REMOVE, file);
37. }
38.
39. int
40. open (const char *file)
41. {
42.     return syscall1 (SYS_OPEN, file);
43. }
44.
45. int
46. filesize (int fd)
47. {
48.     return syscall1 (SYS_FILESIZE, fd);
```

```
49. }
50.
51. int
52. read (int fd, void *buffer, unsigned size)
53. {
54.     return syscall3 (SYS_READ, fd, buffer, size);
55. }
56.
57. int
58. write (int fd, const void *buffer, unsigned size)
59. {
60.     return syscall3 (SYS_WRITE, fd, buffer, size);
61. }
62.
63. void
64. seek (int fd, unsigned position)
65. {
66.     syscall2 (SYS_SEEK, fd, position);
67. }
68.
69. unsigned
70. tell (int fd)
71. {
72.     return syscall1 (SYS_TELL, fd);
73. }
74.
75. void
76. close (int fd)
77. {
78.     syscall1 (SYS_CLOSE, fd);
79. }
80.
81. mapid_t
82. mmap (int fd, void *addr)
83. {
84.     return syscall2 (SYS_MMAP, fd, addr);
85. }
86.
87. void
88. munmap (mapid_t mapid)
89. {
90.     syscall1 (SYS_MUNMAP, mapid);
91. }
92.
93. bool
94. chdir (const char *dir)
95. {
96.     return syscall1 (SYS_CHDIR, dir);
97. }
98.
99. bool
100. mkdir (const char *dir)
101. {
102.     return syscall1 (SYS_MKDIR, dir);
103. }
104.
105. bool
106. readdir (int fd, char name[READDIR_MAX_LEN + 1])
107. {
108.     return syscall2 (SYS_READDIR, fd, name);
109. }
110.
111. bool
112. isdir (int fd)
113. {
114.     return syscall1 (SYS_ISDIR, fd);
115. }
116.
```

```

117. int
118. inumber (int fd)
119. {
120.     return syscall1 (SYS_INNUMBER, fd);
121. }

```

이때 이 macro들이 user mode에서 kernel mode로 전환하는 역할을 수행한다:

```

1. /* Invokes syscall NUMBER, passing no arguments, and returns the
2.    return value as an `int'. */
3. #define syscall0(NUMBER)
4.     ({
5.         int retval;
6.         asm volatile
7.             ("pushl %[number]; int $0x30; addl $4, %%esp"
8.              : "=a" (retval)
9.              : [number] "i" (NUMBER)
10.             : "memory");
11.         retval;
12.     })
13.
14. /* Invokes syscall NUMBER, passing argument ARG0, and returns the
15.    return value as an `int'. */
16. #define syscall1(NUMBER, ARG0)
17.     ({
18.         int retval;
19.         asm volatile
20.             ("pushl %[arg0]; pushl %[number]; int $0x30; addl $8, %%esp" \
21.              : "=a" (retval)
22.              : [number] "i" (NUMBER),
23.              [arg0] "g" (ARG0)
24.              : "memory");
25.         retval;
26.     })
27.
28. /* Invokes syscall NUMBER, passing arguments ARG0 and ARG1, and
29.    returns the return value as an `int'. */
30. #define syscall2(NUMBER, ARG0, ARG1)
31.     ({
32.         int retval;
33.         asm volatile
34.             ("pushl %[arg1]; pushl %[arg0]; "
35.              "pushl %[number]; int $0x30; addl $12, %%esp"
36.              : "=a" (retval)
37.              : [number] "i" (NUMBER),
38.              [arg0] "r" (ARG0),
39.              [arg1] "r" (ARG1)
40.              : "memory");
41.         retval;
42.     })
43.
44. /* Invokes syscall NUMBER, passing arguments ARG0, ARG1, and
45.    ARG2, and returns the return value as an `int'. */
46. #define syscall3(NUMBER, ARG0, ARG1, ARG2)
47.     ({
48.         int retval;
49.         asm volatile
50.             ("pushl %[arg2]; pushl %[arg1]; pushl %[arg0]; "
51.              "pushl %[number]; int $0x30; addl $16, %%esp"
52.              : "=a" (retval)
53.              : [number] "i" (NUMBER),
54.              [arg0] "r" (ARG0),
55.              [arg1] "r" (ARG1),
56.              [arg2] "r" (ARG2)
57.              : "memory");
58.         retval;
59.     })
60.

```


위의 macro는 GCC inline assembly(asm volatile)를 사용하여 저수준의 CPU 명령을 직접 실행한다. 여기서 int \$0x30의 경우 software interrupt 0x30번을 발생시키는 명령어인데, 이 명령이 실행되는 순간 CPU는 현재 실행 중인 user program을 일시 중단하고 미리 등록된 kernel의 interrupt handler로 제어권을 넘김으로써 user mode에서 kernel mode로 전환된다.

2) threads/intr-stubs.S

int \$0x30 명령어가 실행된 직후에 CPU가 가장 먼저 실행하는 kernel code이다. 모든 종류의 interrupt, exception, system call이 kernel로 들어오는 공통된 진입로 역할을 한다. Interrupt 처리 과정은 다음과 같다:

- ① int \$0x30 명령이 실행되면 CPU는 하던 일을 멈춘다.
- ② CPU는 IDT(interrupt Descriptor Table)라는 kernel이 미리 설정해 둔 table을 참조한다.
- ③ IDT의 0x30번 항목에 등록된 코드 주소로 점프하여 실행을 시작한다. Pintos에서는 이 주소가 intr_stubs.S에 정의된 intr30_stub를 가리킨다.

이 파일의 가장 아랫부분에는 STUB macro를 이용해 256개의 작은 코드 조각(intr00_stub ~ intrff_tsub)을 자동으로 생성하는 부분이 있다.

```

1. /* Emits a stub for interrupt vector NUMBER. */
2. #define STUB(NUMBER, TYPE) \
3.     .text; \
4.     .func intr##NUMBER##_stub; \
5.     intr##NUMBER##_stub: \
6.         TYPE; \
7.         push $0x##NUMBER; \
8.         jmp intr_entry; \
9.     .endfunc; \
10. /* ... */
11.
12. /* ... */
13. STUB(30, zero) STUB(31, zero) /* ... */

```

- 모든 intrNN_stub는 intr_entry(C handler 호출 준비를 하는 함수)로 모인다. 이 부분은 interrupt를 처리할 C 함수(intr_handler)를 호출하기 전에 필요한 모든 준비 작업을 수행한다.
- intr_handler의 실행이 끝나면 intr_exit으로 돌아와 user program으로 복귀할 준비를 한다.

syscall.c에서 int \$0x30을 호출했으므로, CPU는 IDT를 통해 intr30_stub로 점프한다.

3) threads/interrupt.c

intr-stubs.S에서 넘어온 제어권을 받아, 어떤 interrupt가 발생했는지 C언어 수준에서 판단하고 그에 맞는 처리 함수(handler)를 호출한다. intr-stubs.S가 모든 interrupt의 저수준 진입로였다면, interrupt.c는 그 진입로를 통과한 interrupt들을 분류하고 분배하는 역할을 한다.

- intr_init(): interrupt 시스템을 초기화한다. (부팅 시 1회 실행된다)
- intr_register_int(): 특정 interrupt 번호에 대해 해당 interrupt가 발생할 경우 특정한 C 함수를 호출하라고 kernel에 공식적으로 등록하는 역할을 한다. 다른 kernel 모듈(timer, 키보드, system call handler)이 이 함수를 사용하여 자신의 처리 함수를 등록한다.
- intr_handler(): intr-stub.S의 intr_entry에서 call 명령을 통해 직접 호출되는 C 함수이다.

4) How a user program calls syscall_handler() in pintos/src/userprog/syscall.c?

① user 라이브러리 함수 호출

예를 들어, 화면에 "hello"를 출력하기 위해 다음과 같은 코드를 실행한다.

```
1. write(1, "hello", 5); // fd=1(stdout), "hello", 5 bytes
```

이 write() 함수는 kernel code가 아닌 lib/user/syscall.c에 정의된 user 라이브러리 함수이다. write() 함수는 내부적으로 인자 3개를 받는 syscall3 매크로를 호출한다.

```
1. // in lib/user/syscall.c
2. int write (int fd, const void *buffer, unsigned size)
3. {
4.     return syscall3 (SYS_WRITE, fd, buffer, size);
5. }
6.
```

② 커널 모드 전환 준비 및 실행 (lib/user/syscall.c의 assembly)

syscall3 매크로는 인라인 assembly를 통해 kernel을 호출하기 위한 준비를 한다.

```
1. syscall3
2. pushl %[arg2];
3. pushl %[arg1];
4. pushl %[arg0];
5. pushl %[number];
6. int $0x30;
7. addl $16, %%esp;
```

Pintos 매뉴얼에서 언급한 80x86 Calling Convention에 따라 system call에 필요한 모든 인자와 system call 번호가 user stack에 push된다. 인자는 역순으로, system call 번호가 가장 마지막에 push된다. 이후 int \$0x30 명령어가 실행된다. 이 명령어는 CPU에게 0x30번 소프트웨어 exception을 발생시킨다. 이 순간, CPU는 하던 일을 멈추고 user mode에서 kernel mode로 전환한 뒤, kernel이 미리 지정해둔 코드를 실행하기 시작한다.

③ Interrupt 저수준 처리

CPU는 `int $0x30` 명령을 받으면, kernel 이 부팅 시 `interrupt.c`의 `intr_init()`에서 설정해 둔 Interrupt Descriptor Table를 참조하여 0x30번 항목에 등록된 `intr30_stub`으로 점프한다. `intr30_stub`은 stack에 dummy 에러 코드(0)와 interrupt 번호(0x30)를 push한다. 이후 모든 stub의 공통 처리 지점인 `intr_entry`로 이동한다. `intr_entry`에서는 `pushal` 등의 명령을 통해 user program의 모든 register 상태(EAX, EBX 등)를 kernel stack에 저장한다. 모든 상태를 저장한 후, assembly code는 `call intr_handler`를 통해 `threads/interrupt.c`에 있는 C 함수를 호출한다.

④ Interrupt 분배 및 최종 handler 호출

`intr-stubs.S`는 `intr_handler()` 함수를 호출하면서 인자로 kernel stack에 저장된 `struct intr_frame`의 pointer를 넘겨준다. 이때 `intr_handler()`는 `intr_frame`에서 interrupt 번호 `vec_no`를 확인한다. 이 값은 0x30이다. `intr_handler()`는 `intr_handlers` 배열의 0x30번 인덱스에 저장된 함수 pointer를 가져온다. 이 pointer는 부팅 시 `syscall_init()`에 의해 `syscall_handler()` 함수를 가리키도록 미리 설정되었다. 이후 `handler(frame)` 코드가 실행되면서, `userprog/syscall.c`의 `syscall_handler()` 함수가 호출된다.

3. Analysis of file system

1) filesystem/file.c

이 파일의 모든 것은 struct file 구조체를 중심으로 동작한다.

```
1. /* An open file. */
2. struct file
3. {
4.     struct inode *inode;           /* File's inode. */
5.     off_t pos;                    /* Current position. */
6.     bool deny_write;              /* Has file_deny_write() been called? */
7. };
```

- struct inode *inode: 이 파일 객체가 가리키는 실제 파일의 메타데이터 및 데이터 블록 정보(inode)에 대한 pointer이다. 파일 크기, disk 상의 위치 정보는 모두 이 inode를 통해 접근한다.
- off_t pos: 파일 내의 현재 위치(offset)을 나타낸다. file_read나 file_write가 호출되면 이 pos 위치부터 데이터를 읽거나 쓴다. 같은 파일을 여러 process가 열더라도, 각 struct file 객체는 자신만의 pos 값을 가지므로 서로 다른 위치를 읽고 쓸 수 있다.
- bool deny_write: 이 파일이 write 금지 상태인지 나타내는 flag이다.

주요 함수는 다음과 같다:

- struct file *file_open(struct inode *inode): open system call은 궁극적으로 이 함수를 호출한다. 주어진 inode를 가지고 새로운 struct file 객체를 memory에 할당하여 생성한다. 이때 pos를 0으로 초기화하고, deny_write를 false로 설정한다.
- struct file *file_reopen(struct file *file): 이미 열려 있는 file 객체와 동일한 inode를 가리키는 새로운 struct file 객체를 생성한다. 내부적으로 inode_reopen()을 호출하여 open 카운트를 증가시키고, file_open을 통해 새로운 file 구조체를 만든다. 이렇게 생성된 두 file 객체는 같은 파일을 가리키지만, pos와 같은 상태는 독립적으로 가진다.
- void file_close(struct file *file): 열려 있는 file 객체를 닫는다. inode_close()를 호출하여 inode의 open 카운트를 감소시키고, struct file 객체가 차지하던 memory를 해제한다.
- off_t file_read(struct file *file, void *buffer, off_t size): read system call이 사용하는 함수로, 현재 위치(file->pos)부터 size 바이트만큼 데이터를 읽어 buffer에 저장한다. 이때 실제 데이터 읽기 작업은 inode_read_at() 함수로 진행하고, 읽은 바이트 수만큼 file->pos를 증가시킨다.
- off_t file_write(struct file *file, const void *buffer, off_t size): write system call이 사용하는 함수로, 현재 위치(file->pos)부터 buffer에 있는 데이터를 size 바이트만큼 파일에 쓴다. 이때 실제 데이터 쓰기 작업은 inode_write_at() 함수로 진행하고, 쓴

바이트 수만큼 file->pos를 증가시킨다.

- off_t file_read_at(...) 및 off_t file_write_at(...): file_read() 및 file_write() 함수와 달리 파일의 현재 위치(file->pos)를 사용하지 않고 명시적으로 주어진 file_ofs 위치에서 읽거나 쓴다. 이때 이 함수들은 file->pos 값을 변경하지 않는다.
- void file_seek(struct file *file, off_t new_pos): seek system call이 사용하는 함수로, 파일의 현재 위치 file->pos를 new_pos로 변경한다.
- off_t file_tell(struct file *file): tell system call이 사용하는 함수로, 파일의 현재 위치 file->pos를 반환한다.
- off_t file_length(struct file *file): filesize system call이 사용하는 함수로, 파일의 전체 크기를 바이트 단위로 반환한다. 실제 크기 정보는 inode에 있으므로 inode_length()를 호출하여 결과를 얻는다.
- void file_deny_write(struct file *file) 및 void file_allow_write(struct file *file): 파일에 대한 write를 금지(file_deny_write()) / 허용(file_allow_write())한다. 내부적으로 inode_deny_write() / inode_allow_write()를 호출하여 inode 레벨에서 write 권한을 제어한다.

file.c에 정의된 함수들은 system call 구현에 직접적으로 사용된다.

- open() system call → filesys_open() → file_open()
- close() system call → file_close()
- read() system call → file_read()
- write() system call → file_write()
- filesize() system call → file_length()
- seek() system call → file_seek()
- tell() system call → file_tell()

2) filesys/filesys.c

filesys.c에 정의된 함수는 다음과 같다:

- void filesys_init(bool format): kernel 부팅 시 호출되어 파일 시스템 전체를 초기화한다.
 - block_get_role(BLOCK_FILESYS)를 통해 파일 시스템으로 사용할 블록 디바이스(디스크 파티션)를 가져온다.
 - 하위 모듈인 inode_init()과 free_map_init()을 호출하여 각각을 초기화한다.
 - format 인자가 true이면 do_format()을 호출하여 disk를 완전히 포맷하고 새로운 파일 시스템을 만든다.

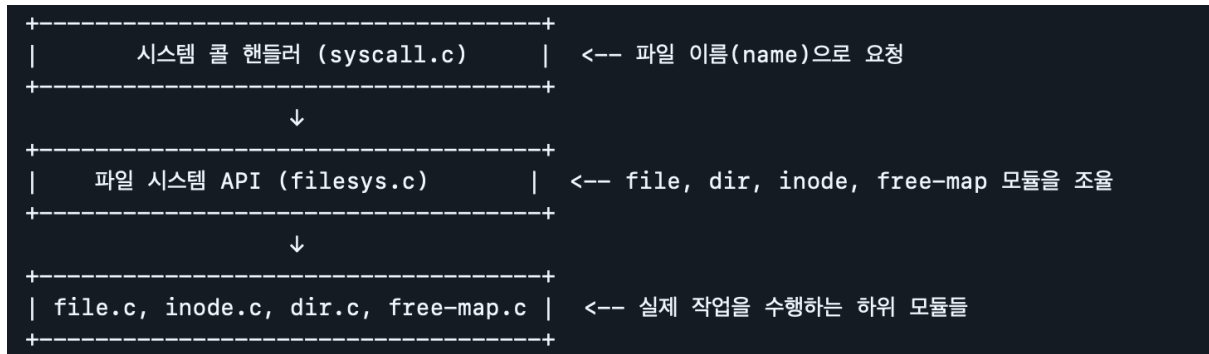
- `free_map_open()`을 호출하여 disk의 free map 정보를 메모리로 읽어들인다.
- `void filesystems_done(void)`: pintos 시스템이 종료될 때 호출되며, `free_map_close()`를 호출하여 변경된 free map 정보(어떤 디스크 블록이 사용 중이고 비어있는지에 대한 정보)를 디스크에 안전하게 저장한다.
- `bool filesystems_create(const char *name, off_t initial_size)`: 이름이 `name`이고 초기 크기가 `initial_size`인 파일을 생성하며, 여러 하위 모듈을 조율하는 작업을 수행한다.
 - `dir_open_root()`: 루트 디렉토리를 연다. (pintos project 2에서는 모든 파일이 루트 디렉토리에 생성된다)
 - `free_map_allocate()`: 파일의 메타데이터를 저장할 inode를 위해 디스크에서 비어 있는 섹터 하나를 할당받는다.
 - `inode_create()`: 할당받은 섹터에 `initial_size` 크기를 갖는 inode를 생성한다.
 - `dir_add()`: 루트 디렉토리에 파일 이름(`name`)과 방금 생성한 inode의 디스크 섹터 위치를 한 쌍으로 하는 엔트리를 추가한다.

위의 과정 중 하나라도 실패하는 경우, `free_map_release()` 등을 통해 지금까지의 작업을 되돌려 일관성을 유지한다.
- `struct file *filesystems_open(const char *name)`: `file.c`와 `dir.c`를 연결하는 함수로, 이름이 `name`인 파일을 연다.
 - `dir_open_root()`: 루트 디렉토리를 연다.
 - `dir_lookup()`: 디렉토리에서 `name`에 해당하는 파일 엔트리를 찾아 그 파일의 inode 정보를 가져온다.
 - `file_open()`: `dir_lookup()`을 통해 얻은 inode 포인터를 `file.c`의 `file_open()` 함수에 전달한다. `file_open()`은 이 inode를 바탕으로 `struct file` 객체를 생성하여 반환한다. 이 객체가 열린 파일을 나타낸다.
- `bool filesystems_remove(const char *name)`: 이름이 `name`인 file을 삭제한다. `dir_open_root()`로 루트 디렉토리를 열고 `dir_remove()`를 호출하여 디렉토리에서 해당 파일 엔트리를 제거하는 작업을 수행한다. `dir_remove()`는 내부적으로 inode와 데이터 블록을 해제하는 과정을 처리한다.

이 파일의 함수들은 파일 관련 system call을 구현할 때 직접적으로 호출된다.

- `create()` system call → `filesystems_create()`
- `open()` system call → `filesystems_open()`
- `remove()` system call → `filesystems_remove()`

또한 `filesystems.c`는 파일 시스템의 계층 구조에서 상위 계층에 위치한다.



3) filesystem/inode.c

inode.c는 파일의 메타데이터와 disk 상의 데이터 블록 위치를 관리하는 역할을 한다.

```
1. /* On-disk inode. Must be exactly BLOCK_SECTOR_SIZE bytes long. */
2. struct inode_disk
3. {
4.     block_sector_t start;      /* First data sector. */
5.     off_t length;             /* File size in bytes. */
6.     unsigned magic;           /* Magic number. */
7.     uint32_t unused[125];      /* Not used. */
8. };
```

struct inode_disk는 파일의 메타데이터를 담아 disk에 직접 저장되는 구조체이다. (disk 상의 inode) 이때 크기는 항상 disk 섹터 크기(512바이트)와 일치해야 한다.

- block_sector_t start: 이 파일의 데이터가 시작되는 첫 번째 데이터 섹터의 번호를 가리킨다. (pintos project 2의 기본 파일 시스템은 파일 데이터가 disk 상에 연속적으로 저장된다고 가정한다)
- off_t length: 파일의 실제 크기(바이트 단위)를 나타낸다.
- unsigned magic: INODE_MAGIC(0x494e4f44) 값을 가지며, 이 섹터가 유효한 inode임을 확인하는데 사용된다.

```
1. /* In-memory inode. */
2. struct inode
3. {
4.     struct list_elem elem;      /* Element in inode list. */
5.     block_sector_t sector;      /* Sector number of disk location. */
6.     int open_cnt;               /* Number of openers. */
7.     bool removed;               /* True if deleted, false otherwise. */
8.     int deny_write_cnt;         /* 0: writes ok, >0: deny writes. */
9.     struct inode_disk data;     /* Inode content. */
10.};
```

struct inode는 disk에 있는 inode가 inode_open()을 통해 열렸을 때 memory에 생성되는 구조체이다. (memory 상의 inode) inode_disk의 복사본(data 멤버)와 함께, 실행 중에 필요한 여러 관리 정보를 포함한다.

- block_sector_t sector: 이 inode 자체가 disk의 몇 번 섹터에 저장되어 있는지를 나타낸다.

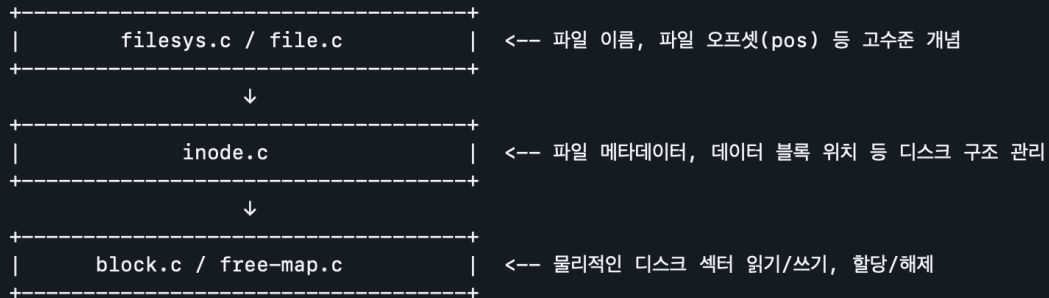
- int open_cnt: 이 inode가 현재 몇 번이나 열렸는지 추적한다. inode_open()을 하면 1 증가하고 inode_close()를 하면 1 감소한다.
- bool removed: 파일이 삭제되었는지 여부를 나타내는 flag이다. True인 경우 open_cnt가 0이 되는 순간 이 inode와 관련된 모든 데이터 블록이 disk에서 해제된다.
- int deny_write_cnt: write 금지 요청이 몇 번 있었는지를 카운트한다. 이 값이 0보다 크면 write가 금지된다.

주요 함수는 다음과 같다:

- bool inode_create(block_sector_t sector, off_t length): fileys_create()에 의해 호출된다. free_map_allocate()를 사용해 length만큼의 데이터 블록을 디스크에서 할당받는다. 또한 inode_disk 구조체를 생성하고, 할당받은 데이터 블록의 시작 주소(start)와 파일 크기(length)를 채운다. 이 inode_disk 구조체를 sector 번호의 disk 위치에 쓴다.
- struct inode *inode_open(block_sector_t sector): sector 번호에 위치한 inode_disk를 disk에서 읽어와, memory 상의 struct node 객체를 생성하여 반환한다. 이때 open_inodes라는 전역 list를 확인하여, 만약 요청된 sector의 inode가 이미 memory에 열려 있다면 disk를 읽지 않고 기존 struct inode의 open_cnt만 증가시켜 반환한다.
- void inode_close(struct inode *inode): open_cnt를 1 감소시킨다. 만약 open_cnt가 0이 되면 이 inode를 더 이상 아무도 사용하지 않는다는 의미이므로 다음을 수행한다:
 - memory에서 struct inode 객체를 제거하고 해제한다.
 - 만약 inode->removed flag가 true였다면 free_map_release()를 호출하여 이 inode가 사용하던 모든 disk 블록(inode 자체와 데이터 블록들)을 해제한다.
- void inode_remove(struct inode *inode): inode->removed flag를 true로 설정하기만 한다. 실제로 disk에서 파일을 즉시 삭제하지 않으며, 실제 삭제는 마지막으로 이 파일을 열고 있던 주체가 inode_close()를 호출하는 시점으로 미뤄진다.
- off_t inode_read_at(struct inode *inode, void *buffer, off_t size, off_t offset): file_read에 의해 호출되며, 파일의 offset 위치에서 size 바이트만큼 데이터를 읽어 buffer에 저장한다. 이때 byte_to_sector()를 통해 offset이 이 disk의 몇 번 섹터에 해당하는지 계산한다. 요청된 size와 offset이 이 섹터 경계와 일치하지 않는 경우(ex. 섹터 중간부터 읽기 시작) bounce buffer라는 임시 메모리 공간을 사용한다.
 - disk에서 섹터 전체를 bounce buffer로 읽어온다.
 - bounce buffer에서 필요한 부분만 buffer로 복사한다.
 이 과정을 필요한 모든 섹터에 대해 반복한다.

- `off_t inode_write_at(...)`: `file_write`에 의해 호출되며, `inode_read_at`과 유사한 방식으로 주어진 `offset`에 데이터를 쓴다. 섹터의 일부만 수정하는 경우 나머지 부분을 보존하기 위해 섹터 전체를 읽어서 수정한 뒤 다시 써야 하므로, 여기에서도 bounce buffer를 사용한다.

`inode.c`는 상위 계층(`file.c`, `fileys.c`)과 physical disk 블록(`block.c`, `free-map.c`) 사이의 다리 역할을 한다.



4. Proposed Design & Implementation

0) Base Code

- struct thread에 변수 추가

```
1. struct thread
2. {
3.     ...
4.
5.     int exit_status;
6.
7.     struct thread *parent;
8.     tid_t parent_tid;
9.     struct list child_list;
10.    struct list_elem child_elem;
11.
12.    bool load_success;
13.
14.    struct semaphore wait_sema;
15.    struct semaphore load_sema;
16.    struct semaphore free_sema;
17.
18. #ifdef USERPROG
19.     /* Owned by userprog/process.c. */
20.     uint32_t *pagedir;          /* Page directory. */
21.
22.     struct file *fd_table[FDT_SIZE];
23.     int next_fd;
24.     struct file *executable_file;
25. #endif
26.     ...
27. };
```

Process 관계 및 상태를 관리하는 멤버 변수는 다음과 같다:

- exit_status: process 종료 시 반환할 종료 상태
- parent: 현재 thread의 parent thread를 가리키는 pointer
- parent_tid: 현재 thread의 parent thread ID
- child_list: 현재 thread의 child thread 목록을 관리하는 리스트 헤더
- child_elem: 현재 thread가 parent의 child_list에 포함될 때 사용되는 리스트 요소
- load_success: 실행 파일(load 함수)의 load 성공 여부 (child process가 load에 실패했는지 parent에게 알리는 데 사용)

Process 동기화 관련 멤버 변수는 다음과 같다:

- wait_sema: child process가 종료될 때 parent process에게 이를 알리기 위해 사용 (SYS_WAIT 구현)
- load_sema: child process가 load를 완료했는지 parent process에게 알리기 위해 사용 (SYS_EXEC 구현)
- free_sema: parent process가 child의 resource 해제를 완료할 때까지 child process의 resource 해제 루틴을 지연시키는 데 사용

User program 관련 멤버 변수는 다음과 같다:

- fd_table[FDT_SIZE]: 현재 process가 열고 있는 파일들을 저장하는 File Descriptor Table (FDT_SIZE는 128로 정의)
- next_fd: 다음으로 할당될 사용 가능한 FD의 번호
- executable_file: 현재 실행 중인 프로그램 파일 자체 (실행 파일 쓰기 금지 기능을 구현하기 위해 필요)
- Child/parent process 관련 추가 및 수정

```

1. static struct thread *find_child_process(tid_t child_tid) {
2.     struct thread *cur = thread_current();
3.     struct list_elem *e;
4.     struct thread *child_t = NULL;
5.
6.     for (e = list_begin(&cur->child_list); e != list_end(&cur->child_list); e = list_next(e))
7.     {
8.         child_t = list_entry(e, struct thread, child_elem);
9.         if (child_t->tid == child_tid) {
10.            return child_t;
11.        }
12.    }
13.    return NULL;
14. }

```

find_child_process() 함수는 현재 thread(cur)의 child 리스트(cur->child_list)를 순회하여 주어진 child_tid를 가진 child thread 구조체를 찾아 반환한다.

```

1. ...
2. char program_name[16];
3. char *save_ptr;
4. strcpy(program_name, file_name, sizeof(program_name));
5. strtok_r(program_name, " ", &save_ptr);
6.
7. tid = thread_create (program_name, PRI_DEFAULT, start_process, fn_copy);
8. ...
9. struct thread *child = get_thread(tid);
10. if (!child) {
11.     return TID_ERROR;
12. }
13.
14. list_push_back(&thread_current()->child_list, &child->child_elem);
15.
16. sema_down(&child->load_sema);
17.
18. if (!child->load_success) {
19.     return TID_ERROR;
20. }
21. return tid;
22.

```

위의 코드는 process_execute() 함수의 일부이다. 새 process를 생성하고 child-parent 관계를 설정하며 load 성공을 대기하는 로직이 추가되었다.

```

1. ...
2. struct thread *cur = thread_current();
3. cur->load_success = success;
4. cur->parent = get_thread(cur->parent_tid);
5.
6. sema_up(&cur->load_sema);
7.
8. ...
9. if (!success) {

```

```

10.     force_exit(-1);
11. }

```

위의 코드는 start_process() 함수의 일부이다. 새 process가 로드된 후 parent에게 상태를 알리는 동기화 로직이 추가되었다.

```

1. int
2. process_wait (tid_t child_tid)
3. {
4.     struct thread *child = find_child_process(child_tid);
5.     if (!child) {
6.         return -1;
7.     }
8.
9.     sema_down(&child->wait_sema);
10.
11.     int status = child->exit_status;
12.     list_remove(&child->child_elem);
13.
14.     sema_up(&child->free_sema);
15.
16.     return status;
17. }

```

process_wait()의 로직은 skeleton code에서 미 구현된 함수이기에 위에 대한 구체적인 구현은 아래에서 설명할 예정이다.

```

1.     ...
2.
3.
4.     struct list_elem *e;
5.     while (!list_empty(&cur->child_list))
6.     {
7.         e = list_pop_front(&cur->child_list);
8.         struct thread *child = list_entry (e, struct thread, child_elem);
9.         if (child) {
10.             sema_up(&child->free_sema);
11.         }
12.     }
13.     ...
14.

```

위의 코드는 process_exit() 함수의 일부이다. process 종료 시 resource를 정리하고 대기 중인 child의 resource 해제를 허용하는 로직이 추가되었다.

1) Process Termination Messages

User process가 정상적으로 exit을 호출하든, exception으로 kernel에 의해 종료되든, 반드시 console에 [process 이름]: exit([종료 코드]) / printf("%s: exit(%d)\n", ...) 형식의 종료 message를 출력해야 한다. message 출력을 위해서는 process 이름과 종료 코드를 알아야 한다.

현재 실행 중인 thread를 종료시키는 force_exit() 함수를 새로 구현하고, 이 함수 내부에서 printf("%s: exit(%d)\n", cur->name, status) 코드를 추가해 termination message를 출력하려 한다. force_exit() 함수의 인자는 종료 코드(status)를 받게끔 설정하려 한다. 이때

process 이름은 process_execute()를 통해 전달된 명령줄에서 첫 번째 단어(실행할 프로그램 이름)만 추출하여 사용하려 한다. 본 design report의 2) Argument Passing에서도 프로그램 이름을 추출하는 방식이 작성되어 있다.

[Implementation]

```
1. void force_exit(int status) {
2.     struct thread *cur = thread_current();
3.     cur->exit_status = status;
4.     printf("%s: exit(%d)\n", cur->name, status);
5.
6.     sema_up(&cur->wait_sema);
7.     sema_down(&cur->free_sema);
8.
9.     thread_exit();
10. }
```

현재 실행 중인 thread를 종료시키는 force_exit() 내에서, 설계한 design을 반영했다. 정상적으로 exit을 호출하거나, exception으로 kernel에 의해 종료되는 등의 어떠한 경우에서든 종료가 되는 상황이 발생하면 (ex. SYS_EXIT, SYS_CLOSE 등) force_exit() 함수를 호출하도록 코드를 작성했다.

2) Argument Passing

기존에는 프로그램 이름만 받을 수 있는 process_execute() 함수를 확장하여, 새로운 process에 명령줄 인자(argument)를 전달할 수 있도록 만드는 것이다. 예를 들어 process_execute("ls -l")을 호출하면 ls 프로그램을 실행하면서 -l이라는 인자를 전달해야 한다. 즉 process_execute() 함수는 이제 단순히 파일 이름 하나가 아닌, 공백으로 구분된 여러 단어를 포함하는 전체 명령줄 문자열을 인자로 받게 된다. 이때 함수는 전달 받은 문자열에 대해 공백을 기준으로 단어 단위로 나눠야 하며, 연속된 여러 개의 공백은 하나의 공백으로 취급해야 한다. 문자열 파싱의 경우, C 라이브러리 함수인 strtok_r()을 사용하려 한다. 그리고 문자열 파싱이 끝난 뒤 가장 중요한 단계는 이 정보들을 새로운 process의 user stack에 올바른 형식으로 배치하는 것이다. Program이 시작될 때 main 함수가 이 인자들을 정상적으로 읽을 수 있도록 stack을 미리 설정해주어야 한다.

process_execute() 함수와 start_process() 함수를 수정함으로써 위의 조건을 달성하고자 한다. process_execute() 함수에서는 malloc_get_page(0)와 strcpy()를 사용하여 전달 받은 file_name의 복사본을 만들고, 이 명령줄 복사본에서 strtok_r을 이용해 첫 번째 단어(실행할 프로그램 이름)만 추출한 뒤, 추출한 프로그램 이름과 파싱하지 않은 전체 명령줄의 복사본을 따로 분리하여 thread_create()에 전달하는 방식으로 구현하려 한다. 이처럼 process_execute()는 프로그램 이름과 전체 명령줄을 분리하고, 실제 stack 구성 작업은 start_process()가 하도록 역할을 분담하려 한다.

start_process()는 새로 생성된 thread에서 실행되는데, 기존에는 단순히 load() 함수만 호출했다. process_execute()로부터 전달 받은 전체 명령줄을 strtok_r 함수를 사용하여 공백 기준으로 자르고, 잘린 각 단어의 시작 주소를 argv 문자열 포인터 배열에 순서대로 저장하며, 총 단어의 개수를 argc에 저장하여 명령줄 파싱을 수행하려 한다. 또한 load()가 성공적으로 실행 파일을 memory에 올린 후, pintos project 2 '3.5.1 Program Startup Details'에 명시된 구조에 맞춰 앞서 저장해놓은 argv와 argc를 활용하여 user stack을 설정하려 한다. 이때 stack은 높은 주소에서 낮은 주소로 쌓인다.

* `"/bin/ls -l foo bar"` 명령줄을 처리하는 과정은 다음과 같다:

1. 문자열을 `"/bin/ls", "-l", "foo", "bar"`로 분리한다.
2. 각 단어를 stack의 top에 저장한다. (이들은 pointer로 참조되므로 실제 저장 순서는 중요하지 않다)
3. 각 문자열에 null pointer를 붙인 address를 stack에 오른쪽에서 왼쪽 순서(`bar → foo → -l → /bin/ls`)로 push한다. 이들은 argv 배열의 요소이며, 마지막 null pointer는 C 표준에 따라 `argv[argc] == NULL`을 만족시킨다. 이 순서는 `argv[0]`이 가장 낮은 virtual address를 갖도록 보장한다.
4. 효율적인 word-aligned 접근을 위해, 첫 push 전에 stack pointer를 4의 배수로 내린다(round down).
5. argv와 argc를 차례로 push한다.
6. 마지막으로 return address를 push한다. Pintos project 2 3.5.1 Program Startup Details를 참고했을 때, user program이 시작되기 전에 stack과 register의 상태는 다음 그림과 같은 형태를 갖는다(`PHYS_BASE = 0xc0000000`, stack pointer 초기값 = `0xbffffffc`):

Address	Name	Data	Type
0xbffffffc	argv[3][...]	'bar\0'	char[4]
0xbffffff8	argv[2][...]	'foo\0'	char[4]
0xbffffff5	argv[1][...]	'-l\0'	char[3]
0xbffffffed	argv[0][...]	'/bin/ls\0'	char[8]
0xbffffffec	word-align	0	uint8_t
0xbffffffe8	argv[4]	0	char *
0xbffffffe4	argv[3]	0xbffffffc	char *
0xbffffffe0	argv[2]	0xbffffff8	char *
0xbffffffdc	argv[1]	0xbffffff5	char *
0xbffffffd8	argv[0]	0xbffffffed	char *
0xbffffffd4	argv	0xbffffffd8	char **
0xbffffffd0	argc	4	int
0xbffffffcc	return address	0	void (*)()

[Implementation]

- process_execute() 함수 수정 - 사용자로부터 "ls -l foo"와 같은 전체 명령줄을 받아 process 생성을 시작하는 진입점이다. 이전에는 단순히 file_name을 thread_create()에 그대로 전달했으며 argument 개념이 없었으나, 설계한 design을 바탕으로 process_execute()를 다음과 같이 수정했다 (프로그램 이름과 전체 명령줄을 분리하여, 실제 stack 구성 작업은 start_process()가 하도록 역할을 분담한다):
 1. malloc_get_page(0)와 strcpy()를 사용하여 전달 받은 file_name의 복사본 fn_copy를 만든다. (strtok_r() 함수가 원본 문자열을 수정하기 때문에, 원본을 보존하고 수정 가능한 복사본을 만들기 위함이다)
 2. strtok_r()을 이용해 복사된 명령줄에서 첫 번째 단어(실행할 프로그램 이름)만 추출한다. 이 이름은 thread의 이름으로 사용된다.
 3. 이후 추출한 프로그램 이름을 전달한다. 이때 파싱하지 않은 전체 명령줄의 복사본(fn_copy)을 전달한다. (thread_create()의 호출 변경)

```

1. tid_t
2. process_execute (const char *file_name)
3. {
4.     char *fn_copy;
5.     tid_t tid;
6.
7.     fn_copy = malloc_get_page (0);
8.     if (fn_copy == NULL)
9.         return TID_ERROR;
10.    strcpy (fn_copy, file_name, PGSIZE);
11.
12.    char program_name[16];
13.    char *save_ptr;
14.    strcpy(program_name, file_name, sizeof(program_name));
15.    strtok_r(program_name, " ", &save_ptr);
16.
17.    tid = thread_create (program_name, PRI_DEFAULT, start_process, fn_copy);
18.
19.    ...
20. }

```

- start_process() 수정
 - process_execute()로부터 전달 받은 전체 명령줄(file_name_)을 strtok_r() 함수를 사용하여 공백 기준으로 자른다. 이때 잘린 각 단어의 시작 주소는 argv 문자열 pointer 배열에 순서대로 저장되고, 총 단어의 개수는 argc에 저장된다.

```

1. char *argv[128];
2. int argc = 0;
3. char *token, *save_ptr;
4. for (token = strtok_r(file_name, " ", &save_ptr); token != NULL;
5.     token = strtok_r(NULL, " ", &save_ptr))
6. {
7.     argv[argc++] = token;
8. }

```

- load()가 성공적으로 실행 파일을 메모리에 올린 후, '3.5.1 Program Startup Details'에 명시된 구조에 맞춰 user stack을 설정한다. Stack은 높은 주소에서 낮은 주소로 쌓인다.

```

1. if (success) {
2.     char *arg_addrs[argc];
3.     int i;
4.
5.     // 1. 실제 인자 문자열 저장 (argv[argc-1]부터 역순으로)
6.     for (i = argc - 1; i >= 0; i--) {
7.         int len = strlen(argv[i]) + 1; // 널 종료 문자 포함
8.         if_esp -= len;
9.         memcpy(if_esp, argv[i], len);
10.        arg_addrs[i] = if_esp; // 스택에 저장된 주소를 임시 저장
11.    }
12.
13.    // 2. Word-align 패딩 (주소를 4의 배수로 맞춤)
14.    while ((int)if_esp % 4 != 0) {
15.        if_esp--;
16.        *(uint8_t *)if_esp = 0;
17.    }
18.
19.    // 3. 인자 문자열의 주소 저장 (argv[argc]부터 역순으로)
20.    if_esp -= 4;
21.    *(char **)if_esp = NULL; // argv[argc] = NULL
22.    for (i = argc - 1; i >= 0; i--) {
23.        if_esp -= 4;
24.        *(char **)if_esp = arg_addrs[i]; // 임시 저장했던 주소 사용
25.    }
26.
27.    // 4. argv 배열의 시작 주소, argc, 가짜 반환 주소 저장
28.    if_esp -= 4;
29.    *(char ***)if_esp = if_esp + 4; // argv (바로 위 주소)
30.    if_esp -= 4;
31.    *(int *)if_esp = argc;
32.    if_esp -= 4;
33.    *(int *)if_esp = 0; // fake return address
34. }

```

3) System Calls

userprog/syscall.c에 system call handler를 구현해야 한다. User program이 kernel의 기능을 요청할 때, 이 handler가 요청을 받아 처리하게 된다. 기존 system call handler는 다음과 같이 작성되어 있으며 구현되어 있지 않은 상태이다:

```

1. static void
2. syscall_handler (struct intr_frame *f UNUSED)
3. {
4.     printf ("system call!\n");
5.     thread_exit ();
6. }

```

system call handler의 개괄적인 동작 방식은 다음과 같다:

- ① user stack에서 system call 번호를 가져온다.
- ② 해당 system call에 필요한 argument들을 stack에서 가져온다.
- ③ 번호에 맞는 적절한 kernel 동작을 수행한다.

이를 위해 우선 syscall handler 내부에 switch문을 구현하여, stack에서 읽어온 system call 번호에 따라 처리를 분기하는 구조로 system call framework를 구현하려 한다. 또한 syscall.c에 safe user memory access를 위한 헬퍼 함수를 추가하려 한다. 이 함수들은 is_user_vaddr와 pagedir_get_page를 활용하여 user가 전달한 모든 포인터가 유효한 user

영역에 속하고 실제 physical memory에 매핑되어 있는지 검사하는 역할을 수행하도록 구현하려 한다. 이때 유효하지 않은 경우, force_exit(-1)을 호출하여 process를 즉시 종료 시키고 kernel을 보호하려 한다. 헬퍼 함수를 간단히 소개하면 다음과 같다:

- static void validate_address(const void *uaddr): 입력 받은 포인터 uaddr이 유효한 사용자 주소인지 확인한다.
- static void validate_buffer(const void *uaddr, unsigned size): 특정 buffer의 시작 주소 uaddr부터 크기 size만큼의 memory가 모두 유효한지 검사한다.
- static void validate_string(const char *uaddr): 널 종료 문자열('\0'로 끝나는 문자열)의 모든 문자 위치가 유효한 사용자 주소인지 검사한다.

또한 file system code는 critical section으로 다루어, 동시에 여러 process가 접근하지 못하도록 보호해야 한다. 이를 위해, syscall.c에 변수 static struct lock filesys_lock을 선언하여 syscall_init에서 lock_init으로 초기화하려 한다. 이후 create, remove, open, read, write 등 file system에 접근하는 모든 system call의 시작과 끝을 lock_acquire(&filesys_lock)과 lock_release(&filesys_lock)으로 감싸려 한다.

```
1. /* System call numbers. */
2. enum
3. {
4.     /* Projects 2 and later. */
5.     SYS_HALT,           /* Halt the operating system. */
6.     SYS_EXIT,          /* Terminate this process. */
7.     SYS_EXEC,          /* Start another process. */
8.     SYS_WAIT,          /* Wait for a child process to die. */
9.     SYS_CREATE,        /* Create a file. */
10.    SYS_REMOVE,        /* Delete a file. */
11.    SYS_OPEN,          /* Open a file. */
12.    SYS_FILESIZE,      /* Obtain a file's size. */
13.    SYS_READ,          /* Read from a file. */
14.    SYS_WRITE,         /* Write to a file. */
15.    SYS_SEEK,          /* Change position in a file. */
16.    SYS_TELL,          /* Report current position in a file. */
17.    SYS_CLOSE,         /* Close a file. */
18. };
```

위와 같이 system call number와 관련된 enum 변수가 pintos/src/lib/syscall-nr.h에 정의되어 있어, 이 변수를 사용하여 system call 동작을 (switch문을 통해) 다음과 같이 정리하여 구현하려 한다:

- SYS_HALT: shutdown_power_off()를 호출하여 pintos를 terminate한다. (shutdown_power_off()의 경우 threads/init.h에 declared되어 있다)
- SYS_EXIT: 현재 프로그램을 종료하고, parent가 wait할 경우를 대비해 status를 kernel에 반환해야 한다.
- SYS_EXEC: child process의 실행 파일 load 성공 여부를 parent가 알 때까지 대기하는 동기화가 필요하다.
- SYS_WAIT: 가장 복잡한 system call로, 직접적인 child만 기다려야 하며 child의 종

료 상태를 정확하게 받아오고 한 번만 기다리게 해야 한다.

- **SYS_CREATE**: file이라는 이름과 initial_size라는 초기 크기를 가진 새로운 파일을 생성한다.
- **SYS_REMOVE**: file이라는 이름의 파일을 삭제한다.
- **SYS_OPEN**: file이라는 이름의 파일을 연다. 각 process는 독립적인 fd 집합을 가진다.
- **SYS_FILESIZE**: fd로 열린 파일의 크기를 바이트 단위로 반환한다.
- **SYS_READ**: fd로 열린 파일에서 size 바이트를 읽어 buffer에 저장한다. 실제 읽은 바이트 수를 반환하며, fd가 0이면 키보드로부터 입력받는다.
- **SYS_WRITE**: buffer의 데이터를 size 바이트만큼 fd 파일에 쓴다. 실제 쓰여진 바이트 수를 반환하며, fd가 1이면 consol에 출력한다. Consol 출력은 여러 process의 출력이 섞이지 않도록 putbuf()를 이용해 한 번에 처리해야 한다.
- **SYS_SEEK**: fd로 열린 파일의 다음 read/write 위치를 파일 시작부터 position 바이트 떨어진 곳으로 변경한다.
- **SYS_TELL**: fd로 열린 파일의 다음 읽기/쓰기 위치를 반환한다.
- **SYS_CLOSE**: 파일 descriptor fd를 닫는다. Process가 종료될 때는 열려 있는 모든 파일이 암묵적으로 닫힌다.

위의 내용을 구현하기 위해 필요한 process control 관련 system call 함수는 다음과 같다:

- halt(): pintos를 완전히 종료시킨다.
- exit(status): 현재 실행 중인 user program을 종료하고, 종료 상태(status)를 kernel에 반환한다. Parent process가 wait으로 이 상태값을 받을 수 있다.
- exec(cmd_line): cmd_line으로 주어진 프로그램을 인자와 함께 실행하고, 새로 만들어진 process의 ID(pid)를 반환한다. 프로그램 load에 실패할 경우 -1을 반환해야 한다. Parent process는 child process의 실행 파일 load 성공 여부를 알 때까지 기다려야 하며, 이를 위해 반드시 synchronization이 필요하다.
- wait(pid): child process pid가 종료될 때까지 기다린 후, child의 종료 상태 값을 받아 반환한다. 이 system call은 pid가 자신의 직접적인 child가 아닐 경우 즉시 -1을 return해야 하고, 이미 wait을 호출했던 pid에 대해 또 호출할 경우 즉시 -1을 return해야 하며, parent가 기다리든 아니든 혹은 parent보다 먼저 끝나는 나중에 끝나는 child process의 모든 resource는 항상 올바르게 해제되어야 한다.

또한 파일 control 및 입출력을 위해 필요한 system call 함수는 다음과 같다:

- create(file, initial_size): initial_size 크기의 file이라는 새 파일을 만든다. 파일을 열지는 않는다.
- remove(file): file을 삭제한다.

- open(file): file을 열고, 파일에 접근할 수 있는 파일 descriptor(fd)를 반환한다. 실패할 경우 -1을 반환한다.
- filesize(fd): fd로 열린 파일의 크기를 바이트 단위로 반환한다.
- read(fd, buffer, size): fd로 열린 파일에서 size 바이트만큼 읽어 buffer에 저장하며, 실제 읽은 바이트 수를 반환한다. fd가 0이면 키보드 입력을 받는다.
- write(fd, buffer, size): buffer의 데이터를 size 바이트만큼 fd로 열린 파일에 쓰며, 실제 쓰여진 바이트 수를 반환한다. fd가 1이면 console에 출력하며, 여러 process의 출력 내용이 섞이지 않도록 한 번의 putbuf() 호출로 처리해야 한다.
- seek(fd, position): fd로 열린 파일에서 다음에 읽거나 쓸 위치를 position으로 이동한다.
- tell(fd): fd로 열린 파일의 현재 위치(다음에 읽거나 쓸 위치)를 반환한다.
- close(fd): fd에 해당하는 파일을 닫는다. Process가 종료되면 열려 있던 모든 파일은 자동으로 닫힌다.

[Implementation]

먼저 syscall_handler() 내부에 다음과 같이 switch문을 구현했다.

```
1. static void
2. syscall_handler (struct intr_frame *f)
3. {
4.     int syscall_number = *(int *)f->esp;
5.
6.     switch (syscall_number){
7.         ...
8.     }
9. }
```

또한 설계한 design을 바탕으로, syscall.c에 validate_address(), validate_buffer(), validate_string() 헬퍼 함수들을 추가했다. 이 함수들을 기반으로 safe user memory access가 가능하도록 구현했다. 각각의 함수 코드는 다음과 같다:

```
1. static void validate_address(const void *uaddr) {
2.     if (uaddr == NULL || !is_user_vaddr(uaddr) || pagedir_get_page(thread_current()->pagedir,
3.         uaddr) == NULL) {
4.         force_exit(-1);
5.     }
6. }
```

위의 함수는 입력 받은 pointer uaddr이 유효한 사용자 address인지 확인한다. uaddr이 NULL이 아닌지 / uaddr이 사용자 address 영역인지 (is_user_vaddr() 함수로 판단) / 현재 thread의 페이지 directory에서 매핑된 physical 페이지가 있는지 (pagedir_get_page()로 판단) 총 세 조건을 검사한다. 이 조건 중 하나라도 만족하지 않으면 강제 종료한다.

```
1. static void validate_buffer(const void *uaddr, unsigned size) {
2.     if (size == 0) return;
3.     validate_address(uaddr);
4.     validate_address(((const char *)uaddr + size - 1));
5. }
```

위의 함수 동작은 설계한 design에서 작성한 내용과 동일하다.

```
1. static void validate_string(const char *uaddr) {
2.     for (;;) uaddr++ {
3.         validate_address(uaddr);
4.         if (*uaddr == '\0')
5.             break;
6.     }
7. }
```

위의 함수 동작은 설계한 design에서 작성한 내용과 동일하다.

Synchronization도 고려해줘야 하는데, file system 코드는 안전하지 않으므로 critical section으로 다루어 동시에 여러 process가 접근하지 못하도록 보호해야 한다. 이를 위해 syscall.c에 정적 변수로 static struct lock filesys_lock;을 선언하고, syscall_init에서 lock_init으로 초기화했다.

```
1. static struct lock filesys_lock;
2.
3. void
4. syscall_init (void)
5. {
6.     intr_register_int (0x30, 3, INTR_ON, syscall_handler, "syscall");
7.     lock_init(&filesys_lock);
8. }
```

create, remove, open, read, write 등 file system에 접근하는 모든 system call의 시작과 끝을 lock_acquire(&filesys_lock)와 lock_release(&filesys_lock)으로 감쌌다. 이를 통해 한 번에 하나의 process만 file system 관련 작업을 수행할 수 있도록 보장한다.

```
1. switch (syscall_number) {
2.     case SYS_CREATE:
3.         ...
4.         validate_string(file_create);
5.         lock_acquire(&filesys_lock);
6.         f->eax = filesys_create(file_create, initial_size);
7.         lock_release(&filesys_lock);
8.         break;
9.     ...
10. }
11.
```

Project 2에서는 아직 virtual memory가 구현되지 않았는데, 이 단계에서 발생하는 모든 page fault는 잠재적으로 심각한 오류이다. 따라서 다음의 조건문을 구현하여 안전장치를 구축했다:

```
1. // userprog/exception.c
2. static void
3. page_fault (struct intr_frame *f)
4. {
5.     ...
6.
7.     if (!user && is_user_vaddr(fault_addr)) {
8.         force_exit(-1);
9.     }
10.    if (user) {
11.        force_exit(-1);
12.    }
13.    ...

```

```
14. }
```

이 두 조건문은 page fault의 원인을 (1) kernel이 system call을 처리하는 도중 사용자의 잘못된 주소를 쓰려다 실패한 경우와 (2) 사용자가 자신의 코드 실행 도중 스스로 잘못된 주소를 쓰려다 실패한 경우로 나누어, 두 경우 모두 잘못을 저지른 해당 process만 종료시키고 kernel은 계속 안정적으로 실행되도록 보장한다. page_fault() 함수는 interrupt.c의 intr_handler() 함수에서 확인하여 호출된다. (frame->vec_no = 14)

Process의 생성, 대기, 종료를 관리하는 system call들에 대해서 다음과 같이 구현했다:

- SYS_HALT: shutdown_power_off()를 호출하는 방식으로 구현했다.

```
1. case SYS_HALT:
2.     shutdown_power_off();
3.     break;
```

- SYS_EXIT: thread 구조체에 int exit_status;를 추가했다. SYS_EXIT handler는 status 값을 thread_current() → exit_status에 저장한 뒤, force_exit을 호출하여 안전하게 process를 parent를 깨우는 등의 모든 종료 절차를 수행한다.

```
1. case SYS_EXIT:
2.     validate_address(f->esp + 4);
3.     int status = *(int *)(f->esp + 4);
4.     if (cur->executable_file != NULL) {
5.         file_allow_write(cur->executable_file);
6.     }
7.     force_exit(status);
8.     break;
9.
10. // threads/thread.h
11. struct thread{
12.     int exit_status;
13. }
```

- SYS_EXEC:

```
1. case SYS_EXEC:
2.     validate_address(f->esp + 4);
3.     const char *cmd_line = *(const char **)(f->esp + 4);
4.     validate_string(cmd_line);
5.     f->eax = process_execute(cmd_line);
6.     break;
```

```
1. // threads/thread.h
2. struct thread{
3.     bool load_success;
4.     struct semaphore load_sema;
5. }
```

- thread 구조체에 struct semaphore load_sema;와 bool load_success;를 추가했다.

```
1. // userprog/process.c
2. tid_t
3. process_execute (const char *file_name)
4. {
5.     ...
6.
7.     struct thread *child = get_thread(tid);
8.     if (!child) {
9.         return TID_ERROR;
10.    }
```

```

11.
12. list_push_back(&thread_current()->child_list, &child->child_elem);
13.
14. sema_down(&child->load_sema);
15.
16. if (!child->load_success) {
17.     return TID_ERROR;
18. }
19.
20. return tid;
21. }
22.
23. // threads/thread.c
24. struct thread *get_thread(tid_t tid) {
25.     struct list_elem *e;
26.     struct thread *t;
27.
28.     for (e = list_begin(&all_list); e != list_end(&all_list); e = list_next(e)) {
29.         t = list_entry(e, struct thread, allelem);
30.         if (t->tid == tid) {
31.             return t;
32.         }
33.     }
34.     return NULL;
35. }

```

- Parent는 process_execute에서 sema_down(&child->load_sema)를 호출하여 child가 load를 마칠 때까지 대기한다. 전체 thread 목록을 검색하여 주어진 tid에 해당하는 struct thread pointer를 찾아주는 get_thread 함수도 추가해주었다.

```

1. // userprog/process.c
2. static void
3. start_process (void *file_name_)
4. {
5.     ...
6.     success = load (argv[0], &if_.eip, &if_.esp);
7.
8.     struct thread *cur = thread_current();
9.     cur->load_success = success;
10.    cur->parent = get_thread(cur->parent_tid);
11.
12.    sema_up(&cur->load_sema);
13.
14.    ...
15. }

```

- Child는 start_process()에서 load 결과를 load_success에 저장한 뒤, sema_up(&child->load_sema)를 호출하여 parent를 깨운다.

● SYS_WAIT:

```

1. case SYS_WAIT:
2.     validate_address(f->esp + 4);
3.     tid_t tid = *(tid_t *) (f->esp + 4);
4.     f->eax = process_wait(tid);
5.     break;

```

```

1. // threads/thread.h
2. struct thread {
3.     struct thread *parent;
4.     tid_t parent_tid;
5.     struct list child_list;
6.     struct list_elem child_elem;
7.
8.     struct semaphore wait_sema;
9.     struct semaphore free_sema;

```

10. }

- Thread 구조체에 parent-child 관계(struct thread *parent, struct list child_list)와 대기/종료 동기화를 위한 semaphore(wait_sema, free_sema)를 추가했다.

```
1. int
2. process_wait (tid_t child_tid)
3. {
4.     struct thread *child = find_child_process(child_tid);
5.     if (!child) {
6.         return -1;
7.     }
8.
9.     sema_down(&child->wait_sema);
10.
11.     int status = child->exit_status;
12.     list_remove(&child->child_elem);
13.
14.     sema_up(&child->free_sema);
15.
16.     return status;
17. }
18.
19. static struct thread *find_child_process(tid_t child_tid) {
20.     struct thread *cur = thread_current();
21.     struct list_elem *e;
22.     struct thread *child_t = NULL;
23.
24.     for (e = list_begin(&cur->child_list); e != list_end(&cur->child_list); e = list_next(e))
25.     {
26.         child_t = list_entry(e, struct thread, child_elem);
27.         if (child_t->tid == child_tid) {
28.             return child_t;
29.         }
30.     }
31.     return NULL;
32. }
33. void force_exit(int status) {
34.     struct thread *cur = thread_current();
35.     cur->exit_status = status;
36.     printf("%s: exit(%d)\n", cur->name, status);
37.
38.     sema_up(&cur->wait_sema);
39.     sema_down(&cur->free_sema);
40.
41.     thread_exit();
42. }
43.
```

- process_wait() 함수는 find_child_process를 통해 자신의 child_list만 탐색한다. Parent는 sema_down(&child->wait_sema)를 호출하여 child가 종료될 때까지 대기한다.
- Child는 exit 호출 시 force_exit() 내에서 sema_up(&cur->wait_sema)를 호출하여 대기 중인 parent를 깨운다. Wait이 성공적으로 끝나면 list_remove(&child->child_elem)를 통해 child list에서 해당 child를 제거함으로써, 한 번만 기다릴 수 있다는 요구사항을 만족시키도록 구현했다.

File 입출력을 위한 system call들은 다음과 같이 구현했다:

- thread 구조체에 struct file *fd_table[FDT_SIZE]; 배열과 int next_fd;를 추가하여 process별로 독립적인 file 관리 table을 구현했다.

```
1. struct thread{
2. #ifndef USERPROG
3.     struct file *fd_table[FDT_SIZE];
4.     int next_fd;
5. #endif
6. }
```

- SYS_CREATE: 인자들의 유효성을 검사한 후, filesys_lock을 획득하여 file system 접근을 동기화한다. 그 후 filesys_create() 함수를 호출하여 실제 파일 생성을 수행하고 결과를 eax register에 저장하여 반환한다.

```
1. case SYS_CREATE:
2.     validate_address(f->esp + 4);
3.     validate_address(f->esp + 8);
4.     const char *file_create = *(const char **)(f->esp + 4);
5.     unsigned initial_size = *(unsigned *)(f->esp + 8);
6.     validate_string(file_create);
7.     lock_acquire(&filesys_lock);
8.     f->eax = filesys_create(file_create, initial_size);
9.     lock_release(&filesys_lock);
10.    break;
```

- SYS_REMOVE: filesys_lock으로 보호된 critical section 안에서 filesys_remove() 함수를 호출하여 파일을 삭제하고, 그 결과를 eax register에 담아 반환한다.

```
1. case SYS_REMOVE:
2.     validate_address(f->esp + 4);
3.     const char *file_remove = *(const char **)(f->esp + 4);
4.     validate_string(file_remove);
5.     lock_acquire(&filesys_lock);
6.     f->eax = filesys_remove(file_remove);
7.     lock_release(&filesys_lock);
8.     break;
```

- SYS_OPEN: filesys_open()으로 파일을 연 후, fd_table에서 2번부터 시작하는 비어 있는 slot을 찾아 file 객체 pointer를 저장하고, 해당 slot의 인덱스를 fd 값으로 eax register에 넣어 반환한다.

```
1. case SYS_OPEN:
2.     validate_address(f->esp + 4);
3.     const char *file_open = *(const char **)(f->esp + 4);
4.     validate_string(file_open);
5.
6.     lock_acquire(&filesys_lock);
7.     struct file *file_obj = filesys_open(file_open);
8.     lock_release(&filesys_lock);
9.
10.    if (file_obj == NULL) {
11.        f->eax = -1;
12.    } else {
13.        struct thread *cur = thread_current();
14.        if (cur->next_fd < FDT_SIZE) {
15.            cur->fd_table[cur->next_fd] = file_obj;
16.            f->eax = cur->next_fd;
17.            cur->next_fd++;
18.        } else {
19.            file_close(file_obj);
20.            f->eax = -1;
```



```

21.     }
22. }
23. break;

```

- SYS_FILESIZE: 전달받은 fd가 유효한 범위(2 이상)에 있는지 확인한다. 유효하다면 fd_table[fd]를 통해 file 객체를 얻고, filesys_lock으로 보호하며 file_length() 함수를 호출해 파일 크기를 얻어 eax register로 반환한다.

```

1. case SYS_FILESIZE:
2.     validate_address(f->esp + 4);
3.     int fd_size = *(int *)(f->esp + 4);
4.     if (fd_size < 2 || fd_size >= FDT_SIZE) {
5.         f->eax = -1;
6.     } else {
7.         struct thread *cur = thread_current();
8.         if (cur->fd_table[fd_size] == NULL) {
9.             f->eax = -1;
10.        } else {
11.            lock_acquire(&filesys_lock);
12.            f->eax = file_length(cur->fd_table[fd_size]);
13.            lock_release(&filesys_lock);
14.        }
15.    }
16.    break;

```

- SYS_READ: fd가 0일 경우 input_getc()를 반복 호출하여 키보드 입력을 처리한다. 다른 fd의 경우, fd_table[fd]에서 파일 객체를 찾아 filesys_lock 보호 하에 file_read()를 호출한다. Buffer의 유효성은 validate_buffer() 함수로 검사한다.

```

1. case SYS_READ:
2.     validate_address(f->esp + 4);
3.     validate_address(f->esp + 8);
4.     validate_address(f->esp + 12);
5.     int fd_read = *(int *)(f->esp + 4);
6.     void *buffer_read = *(void **)(f->esp + 8);
7.     unsigned size_read = *(unsigned *)(f->esp + 12);
8.     validate_buffer(buffer_read, size_read);
9.
10.    if (fd_read == 0) {
11.        unsigned i;
12.        uint8_t *local_buffer = (uint8_t *) buffer_read;
13.        for (i = 0; i < size_read; i++) {
14.            local_buffer[i] = input_getc();
15.        }
16.        f->eax = i;
17.    } else if (fd_read < 2 || fd_read >= FDT_SIZE) {
18.        f->eax = -1;
19.    } else {
20.        struct thread *cur = thread_current();
21.        if (cur->fd_table[fd_read] == NULL) {
22.            f->eax = -1;
23.        } else {
24.            lock_acquire(&filesys_lock);
25.            f->eax = file_read(cur->fd_table[fd_read], buffer_read, size_read);
26.            lock_release(&filesys_lock);
27.        }
28.    }
29.    break;

```

- SYS_WRITE: fd가 1일 경우 putbuf()를 호출하여 요구사항대로 consol 출력을 처리한다. 다른 fd의 경우, fd_table[fd]에서 파일 객체를 찾아 filesys_lock 보호 하에 file_write()를 호출하고 실제 쓰여진 바이트 수를 반환한다.

```

1. case SYS_WRITE:
2.     validate_address(f->esp + 4);
3.     validate_address(f->esp + 8);
4.     validate_address(f->esp + 12);
5.     int fd_write = *(int *)(f->esp + 4);
6.     const void *buffer_write = *(const void **)(f->esp + 8);
7.     unsigned size_write = *(unsigned *)(f->esp + 12);
8.     validate_buffer(buffer_write, size_write);
9.
10.    if (fd_write == 1) { // STDOUT
11.        putchar(buffer_write, size_write);
12.        f->eax = size_write;
13.    } else if (fd_write < 2 || fd_write >= FDT_SIZE) { // 잘못된 fd
14.        f->eax = -1;
15.    } else {
16.        struct thread *cur = thread_current();
17.        struct file *file_to_write = cur->fd_table[fd_write];
18.        if (file_to_write == NULL) { // 닫혔거나 없는 fd
19.            f->eax = -1;
20.        } else {
21.            // file_write 가 알아서 deny_write_cnt 를 확인.
22.            lock_acquire(&filesys_lock);
23.            f->eax = file_write(file_to_write, buffer_write, size_write);
24.            lock_release(&filesys_lock);
25.        }
26.    }
27.    break;

```

- SYS_SEEK: fd_table[fd]에서 파일 객체를 찾은 후, filesys_lock으로 보호된 상태에서 file_seek() 함수를 호출하여 파일의 위치(offset)를 변경한다.

```

1. case SYS_SEEK:
2.     validate_address(f->esp + 4);
3.     validate_address(f->esp + 8);
4.     int fd_seek = *(int *)(f->esp + 4);
5.     unsigned position = *(unsigned *)(f->esp + 8);
6.     if (fd_seek >= 2 && fd_seek < FDT_SIZE) {
7.         struct thread *cur = thread_current();
8.         if (cur->fd_table[fd_seek] != NULL) {
9.             lock_acquire(&filesys_lock);
10.            file_seek(cur->fd_table[fd_seek], position);
11.            lock_release(&filesys_lock);
12.        }
13.    }
14.    break;

```

- SYS_TELL: fd_table[fd]에서 파일 객체를 찾은 후, filesys_lock 보호 하에 file_tell() 함수를 호출하여 현재 위치를 얻고, 그 값을 eax register로 반환한다.

```

1. case SYS_TELL:
2.     validate_address(f->esp + 4);
3.     int fd_tell = *(int *)(f->esp + 4);
4.     if (fd_tell < 2 || fd_tell >= FDT_SIZE) {
5.         f->eax = -1;
6.     } else {
7.         struct thread *cur = thread_current();
8.         if (cur->fd_table[fd_tell] == NULL) {
9.             f->eax = -1;
10.        } else {
11.            lock_acquire(&filesys_lock);
12.            f->eax = file_tell(cur->fd_table[fd_tell]);
13.            lock_release(&filesys_lock);
14.        }
15.    }
16.    break;

```

- SYS_CLOSE: fd_table[fd]에서 파일 객체를 찾아 file_close()를 호출한다. 이후 해당 fd_table slot을 NULL로 설정하여 fd를 다시 사용할 수 있도록 만든다.

```

1. case SYS_CLOSE:
2.     validate_address(f->esp + 4);
3.     int fd_close = *(int *)(f->esp + 4);
4.     if (fd_close < 2 || fd_close >= FDT_SIZE) {
5.         force_exit(-1);
6.     }
7.     if (cur->fd_table[fd_close] == NULL) {
8.         force_exit(-1);
9.     }
10.    lock_acquire(&filesys_lock);
11.    file_close(cur->fd_table[fd_close]);
12.    lock_release(&filesys_lock);
13.    cur->fd_table[fd_close] = NULL;
14.    break;

```

4) Denying Writes to Executables

기존 pintos는 실행 중인 user process의 파일에 다른 process가 수정(write)하는 것을 막는 부분이 구현되어 있지 않다. process가 실행 중일 때, 해당 process의 실행 파일 자체는 write될 수 없도록 막아야 한다. 이를 위해 pintos가 제공하는 file_deny_write() 함수를 사용해야 하며, 파일에 대한 write를 다시 허용하려면 file_allow_write()를 호출하거나 파일을 닫아야(close) 한다. 결과적으로 이 기능을 구현하기 위해서는 process의 실행 파일을 process가 살아 있는 동안 계속 열어 두어야 한다.

Process의 생성(load) 시점과 종료(exit) 시점의 로직을 수정하려 구현하려 한다. Process 생성 시 write를 방지하기 위해, load()에서 filesys_open()으로 실행 파일을 성공적으로 연결 후 file_deny_write(file)을 호출하여 다른 process가 파일에 쓰는 것을 차단하려 한다. 또한 process 종료 시 write를 허용해야 하기 때문에, process_exit()에서 함수 내에서 실행 파일이 열려 있는지 확인하고 파일을 닫는 로직을 추가하려 한다.

[Implementation]

- Process 생성 시 write 방지: 설계한 design을 기반으로 다른 process가 파일에 write하는 것을 차단한다. 이후 열린 파일의 struct file pointer를, thread 구조체에 새로 추가한 executable_file 멤버에 저장한다. 단, load에 실패하고 파일이 열려 있으면 다시 닫아준다.

```

1. // userprog/process.c
2. bool
3. load (const char *file_name, void (**eip) (void), void **esp)
4. {
5.     ...
6.
7.     file_deny_write(file);
8.     t->executable_file = file;
9.
10.    success = true;

```

```

11.
12. ...
13.
14. done:
15. /* We arrive here whether the load is successful or not. */
16. if (!success && file != NULL) {
17.     file_close(file);
18.     t->executable_file = NULL;
19. }
20. return success;
21. }

```

- Process 종료 시 write 허용: 설계한 design을 바탕으로 구현을 진행했다.

```

1. // userprog/process.c
2. void
3. process_exit (void)
4. {
5.     struct thread *cur = thread_current ();
6.     uint32_t *pd;
7.
8.     if (cur->executable_file != NULL) {
9.         file_close(cur->executable_file);
10.        cur->executable_file = NULL;
11.    }
12.    ...
13. }

```

5. Discussion

1) What is different from your design report and why did you take this change?

● Process Termination Messages

- 초기 설계 design과 실제 implementation이 동일하다. 특히 `force_exit()` 함수의 인자를 종료 코드 `status`를 받도록 구현한 부분과, `process_execute()` 함수를 통해 전달된 명령줄에서 실행할 프로그램의 이름인 첫 번째 단어를 추출하여 `process` 이름을 설정한 부분을 design에서 잘 설계해 놓은 덕분에, design을 기반으로 implementation을 잘 수행할 수 있었다.

● Argument Passing

- `process_execute()` 함수와 `start_process()` 함수를 수정하고 두 함수의 역할을 명확히 분담(`process_execute()`의 경우 프로그램 이름과 전체 명령줄을 분리하고, `start_process()`는 실제 stack 구성 작업을 수행)하도록 하여, 명령줄 문자열에 대한 argument passing을 수행하도록 하는 전반적인 흐름이 초기 설계 design과 실제 implementation에서 모두 동일하다. `process_execute()`의 경우 초기 설계 design의 로직을 그대로 구현하였으며, `start_process()` 또한 초기 설계 design의 로직을 따라 구현했다. 이때 `start_process()`의 경우 `load()`가 성공적으로 실행 파일을 메모리에 올린 후 user stack을 설정하는 과정을 design에서는 간략히 서술했으나, 실제 implementation에서는 다음과 같이 구체화하여 구현했다:

1. 실제 인자 문자열 저장 (`argv[argc-1]`부터 역순으로)
2. Word-align 패딩 (주소를 4의 배수로 맞춤)
3. 인자 문자열의 주소 저장 (`argv[argc]`부터 역순으로)
4. `argv` 배열의 시작 주소, `argc`, 가짜 반환 주소 저장

● System Calls

- 초기 설계 design에서 언급한 syscall handler 내부에 switch문을 구현하는 부분과 safe user memory access를 위한 헬퍼 함수를 추가하는 부분은 실제 implementation에서 동일하게 구현하였다. 또한 system call 동작과 이 동작을 구현하기 위해 필요한 process control 관련 함수 및 파일 control & 입출력을 위해 필요한 함수에 대해, 초기 설계 design에서는 함수가 어떤 식으로 동작해야 하는지에 대해 개괄적으로 서술했다. (file system에 접근하는 모든 system call의 시작과 끝을 `lock_acquire()` 및 `lock_release()`로 감싸는 로직 또한 동일하며, 전체적인 흐름은 design에 맞춰 구현했다) 실제 implementation에서는 design에서 일일이 작성할 수 없었던 코드상의 디테일 및 지엽적인 구현 과정이 있었고, 이러한 부분이 코드에 담겨져 있다. 그러나 전반적인 로직 및 흐름은 design과 동일하다.

● Denying Writes to Executables

- Process의 load 시점과 exit 시점의 로직을 수정하여, 실행 중인 user process의 파일에 다른 process가 write하는 것을 막도록 한 부분이 초기 설계 design과 실제 implementation에서 동일하다. Implementation에서는 추가적으로, 다른 process가 write하는 것을 차단한 뒤 열려 있는 파일의 struct file pointer를 executable_file 멤버에 저장하는 로직을 새로 추가해주었다.

2) What have you learned?

전반적으로 user process를 다루는 과정과 system call을 호출하는 과정에 대해서 직접 코드를 구현하며 배울 수 있었다. exec()와 wait()를 통해 parent-child process 관계와 synchronization 문제에 대해 살펴볼 수 있었고, argument passing 구현을 통해 stack 구조를 argv, argc 등을 기반으로 메모리 배치 또한 코드로 구현해보는 과정을 진행할 수 있었다. 또한 file system 관련 system call(open, close, read, write, remove 등)을 직접 구현하며 디테일한 부분에 대해서도 살펴볼 수 있었다. 특히 pintos2에서는 argument passing과 system call 부분에서의 구체적인 코드 구현(Process의 생성, 대기, 종료를 관리하는 system call 및 File 입출력을 위한 system call)을 하며, 초기 설계 design에서 다 담아내지 못했던 동작 로직을 직접 파악해볼 수 있었다.

Pintos project 1 과정에서도 느꼈지만, 특히 pintos project 2 수행 과정에서는 미리 설계한 design을 기반으로 대부분의 implementation을 잘 수행할 수 있었다. Skeleton code의 구체적인 analysis를 기반으로 체계적인 design을 설계하는 것이 코드 작성 및 전반적인 project 진행에 있어 매우 중요하다는 것을 다시 한 번 실감하는 계기가 되었다. 이후 가장 난이도가 높은 pintos project 3에서도 원활한 코드 작성 및 project 진행을 위해 analysis와 design을 꼼꼼하게 수행할 예정이다.

3) What are the limitations of your implementation?

Pintos project 2에서 구현한 코드가 모든 test를 전부 통과했으나, 몇 가지 한계점을 분석해볼 수 있다. 우선 fd_table의 크기를 고정 값(#define FDT_SIZE)으로 정해 확장성이 다소 떨어질 수 있다. 또한 validate_address(), validate_buffer()는 system call이 발생할 때마다 매번 pagedir 검사를 수행하기 때문에, 런타임 성능에 다소 부담이 될 수 있다. 이외에도 pintos project 2에서는 모든 파일에 대한 연산을 전역 filesys_lock으로 직렬화해 구현하여, multi-thread 환경에서 병렬성이 떨어진다는 한계점이 존재한다.

4) Test Result

```
pass tests/userprog/exec-bound
pass tests/userprog/exec-bound-2
pass tests/userprog/exec-bound-3
pass tests/userprog/exec-multiple
pass tests/userprog/exec-missing
pass tests/userprog/exec-bad-ptr
pass tests/userprog/wait-simple
pass tests/userprog/wait-twice
pass tests/userprog/wait-killed
pass tests/userprog/wait-bad-pid
pass tests/userprog/multi-recurse
pass tests/userprog/multi-child-fd
pass tests/userprog/rox-simple
pass tests/userprog/rox-child
pass tests/userprog/rox-multichild
pass tests/userprog/bad-read
pass tests/userprog/bad-write
pass tests/userprog/bad-read2
pass tests/userprog/bad-write2
pass tests/userprog/bad-jump
pass tests/userprog/bad-jump2
pass tests/userprog/no-vm/multi-oom
pass tests/filesys/base/lg-create
pass tests/filesys/base/lg-full
pass tests/filesys/base/lg-random
pass tests/filesys/base/lg-seq-block
pass tests/filesys/base/lg-seq-random
pass tests/filesys/base/sm-create
pass tests/filesys/base/sm-full
pass tests/filesys/base/sm-random
pass tests/filesys/base/sm-seq-block
pass tests/filesys/base/sm-seq-random
pass tests/filesys/base/syn-read
pass tests/filesys/base/syn-remove
pass tests/filesys/base/syn-write
All 80 tests passed.
make[1]: Leaving directory '/home/stu12/pintos/src/userprog/build'
stu12@cse-edu:~/pintos/src/userprog$ git branch
  main
  master
  pintos1
  pintos2
  project1
* project2
```

서버의 project2 branch에서 테스트를 10회 이상 진행한 결과, 80개의 테스트 모두 통과하는 것을 확인하였다.